Année de Formation 2025/2026

M205 : Développer en back end
TP : Migration

Exercice 1 : Création d'une table basique

Objectif : Créer une migration pour une table articles avec les colonnes suivantes :

- id : clé primaire, auto-incrémentée
- title : chaîne de caractères, longueur maximale de 255
- content : texte, nullable
- created_at et updated_at : gérés automatiquement par Laravel

Étapes :

1. Créez la migration avec la commande Artisan.
2. Modifiez la migration pour ajouter les colonnes requises.
3. Exécutez la migration pour créer la table dans la base de données.

Solution :

```
php artisan make:migration create_articles_table
```

Dans la migration

```
Schema::create('articles', function (Blueprint $table) {
        $table->id();
        $table->string('title', 255);
        $table->text('content')->nullable();
        $table->timestamps();
    });
```

Exercice 2 : Modification d'une table existante

Objectif : Ajouter une colonne author de type chaîne de caractères (longueur de 100) à la table articles.

Étapes :

1. Créez une nouvelle migration pour ajouter la colonne author.
2. Ajoutez la logique pour modifier la table dans cette migration.
3. Exécutez la migration pour appliquer le changement.

Solution :

```
php artisan make:migration add_author_to_articles_table --table=articles
```

Dans la migration :

```
Schema::table('articles', function (Blueprint $table) {
        $table->string('author', 100)->nullable();
    });
```

Exercice 3 : Ajout de relations avec des clés étrangères

Objectif : Créer une table comments qui contient un commentaire associé à un article. La table doit inclure les colonnes suivantes :

- id : clé primaire, auto-incrémentée
- article_id : clé étrangère vers la table articles
- comment : texte du commentaire
- created_at et updated_at : gérés automatiquement par Laravel

Étapes :

1. Créez la migration pour la table comments.
2. Définissez la clé étrangère avec la contrainte pour article_id.
3. Exécutez la migration pour créer la table.

Solution :

```
php artisan make:migration create_comments_table
```

Dans la migration :

```
Schema::create('comments', function (Blueprint $table) {
        $table->id();
        $table->foreignId('article_id')->constrained('articles')-
>onDelete('cascade');
        $table->text('comment');
        $table->timestamps();
    });
```

Exercice 4 : Suppression d'une colonne

Objectif : Supprimer la colonne author de la table articles.

Étapes :

1. Créez une migration pour supprimer la colonne author.
2. Ajoutez la logique pour supprimer la colonne.
3. Exécutez la migration.

Solution :

```
php artisan make:migration drop_author_from_articles_table --table=articles
```

Dans la migration :

```
Schema::table('articles', function (Blueprint $table) {
        $table->dropColumn('author');
    });
```

Exercice 5 : Modification d'une colonne

Objectif : Modifier la colonne title de la table articles pour qu'elle ait une longueur maximale de 150 caractères.

Étapes :

1. Créez une migration pour modifier la colonne.
2. Utilisez la méthode adéquate pour redimensionner la colonne title.
3. Exécutez la migration.

Solution :

```
php artisan make:migration modify_title_in_articles_table --table=articles
```

Dans la migration :

```
Schema::table('articles', function (Blueprint $table) {
        $table->string('title', 150)->change();
    });
```

Exercice 6 : Création de tables avec une relation "One-to-One"

Objectif : Créer deux tables users et profiles avec une relation "one-to-one". Chaque utilisateur doit avoir un profil unique, et chaque profil doit appartenir à un seul utilisateur.

Détails des tables :

1. **Table users** :
   - id : clé primaire, auto-incrémentée
   - name : chaîne de caractères, longueur maximale de 100
   - email : chaîne de caractères, longueur maximale de 150, unique
2. **Table profiles** :
   - id : clé primaire, auto-incrémentée
   - user_id : clé étrangère vers users
   - phone : chaîne de caractères, longueur maximale de 15, nullable
   - address : chaîne de caractères, longueur maximale de 255, nullable
   - birthdate : date, nullable

Étapes :

1. Créez les migrations pour les tables users et profiles.
2. Ajoutez la clé étrangère user_id dans la table profiles pour établir la relation "one-to-one".
3. Exécutez les migrations pour appliquer les changements.

Solution :

Créez les migrations avec les commandes suivantes :

```
php artisan make:migration create_users_table
php artisan make:migration create_profiles_table
```

Dans la migration create_users_table :

```
Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name', 100);
        $table->string('email', 150)->unique();
        $table->timestamps();
    });
```

Dans la migration create_profiles_table :

```
Schema::create('profiles', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained('users')-
>onDelete('cascade');
        $table->string('phone', 15)->nullable();
        $table->string('address', 255)->nullable();
        $table->date('birthdate')->nullable();
        $table->timestamps();
    });
```

Exercice 7 : Création de tables avec une relation "One-to-Many"

Objectif : Créer deux tables categories et products avec une relation "one-to-many". Chaque produit doit appartenir à une seule catégorie, mais chaque catégorie peut contenir plusieurs produits.

Détails des tables :

1. **Table categories** :
    - id : clé primaire, auto-incrémentée
    - name : chaîne de caractères, longueur maximale de 100, unique
    - description : texte, nullable
2. **Table products** :

- id : clé primaire, auto-incrémentée
- category_id : clé étrangère vers categories
- name : chaîne de caractères, longueur maximale de 150
- price : décimal, précision de 8,2 (ex. 999,99)
- stock : entier, représente le nombre d'unités en stock

Étapes :

1. Créez les migrations pour les tables categories et products.
2. Définissez la clé étrangère category_id dans la table products pour établir la relation.
3. Exécutez les migrations pour appliquer les changements.

Solution :

Créez les migrations avec les commandes suivantes :

```
php artisan make:migration create_categories_table
php artisan make:migration create_products_table
```

Dans la migration create_categories_table :

```
Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name', 100)->unique();
        $table->text('description')->nullable();
        $table->timestamps();
    });
```

Dans la migration create_products_table :

```
Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->foreignId('category_id')-
>constrained('categories')->onDelete('cascade');
        $table->string('name', 150);
        $table->decimal('price', 8, 2);
        $table->integer('stock');
        $table->timestamps();
    });
```

Exercice 8 : Création de tables avec une relation "Many-to-Many"

Objectif : Mettre en place une relation de type "many-to-many" entre les tables students et courses à l'aide d'une table pivot course_student.

1. **Table students** :
   - id : clé primaire, auto-incrémentée
   - name : chaîne de caractères, longueur maximale de 100
   - email : chaîne de caractères, longueur maximale de 150, unique

2. **Table courses** :
    - id : clé primaire, auto-incrémentée
    - title : chaîne de caractères, longueur maximale de 150
    - description : texte, nullable
3. **Table pivot course_student** :
    - student_id : clé étrangère vers students
    - course_id : clé étrangère vers courses
    - enrolled_at : date de l'inscription

Étapes :

1. Créez les migrations pour les tables students, courses et course_student.
2. Définissez les relations dans les migrations pour que student_id et course_id soient des clés étrangères et soient supprimées si un étudiant ou un cours est supprimé.
3. Exécutez toutes les migrations.

Solution :

Créez les migrations avec les commandes suivantes :

```
php artisan make:migration create_students_table
php artisan make:migration create_courses_table
php artisan make:migration create_course_student_table
```

Dans la migration create_students_table

```
Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name', 100);
        $table->string('email', 150)->unique();
        $table->timestamps();
    });
```

Dans la migration create_courses_table :

```
Schema::create('courses', function (Blueprint $table) {
        $table->id();
        $table->string('title', 150);
        $table->text('description')->nullable();
        $table->timestamps();
    });
```

Dans la migration create_course_student_table :

```
Schema::create('course_student', function (Blueprint $table) {
        $table->foreignId('student_id')->constrained('students')-
>onDelete('cascade');
        $table->foreignId('course_id')->constrained('courses')-
>onDelete('cascade');
        $table->date('enrolled_at');
        $table->timestamps();
    });
```